

Data Structures & Algorithms

Solutions

Sample Exam Questions

Data Structures & Algorithms

1. One can, inefficiently, implement a queue using two stacks, without using any further data structures such as arrays, linked lists etc.

- (a) Explain the basic approach to implementing a queue this way. [5 marks]
- (b) Write the pseudocode for the enqueue function. [5 marks]
- (c) Write the pseudocode for the dequeue function. [5 marks]
- (d) Analyse, in terms of $O(g(n))$, the complexity of each operation and explain, briefly, your reasoning. [5 marks]

Topics: 02 - Stacks and Queues, 03 - Efficiency and Complexity

2. The handout document for this module (page 37) gives the pseudocode for a recursive version of a function to calculate the size of a binary tree.

```
1 size(t)
2 {
3     if ( isEmpty(t) )
4         return 0
5     else
6         return (1 + size(left(t)) + size(right(t)))
7 }
```

Write a non-recursive pseudocode version using a stack to keep track of the work still to be done. [20 marks]

Topics: 04 - Trees

3. Construct a **minimal** AVL tree of positive integers and of height 5

- (a) Draw that AVL tree. [5 marks]
- (b) Identify a value that on insertion into the tree would not trigger any rotation. [5 marks]
- (c) Identify a value that on insertion into the tree would trigger precisely one rotation. [5 marks]
- (d) Identify an insertion into that tree that would trigger precisely one double rotation, i.e. a right rotation followed by a left rotation or a left rotation followed by a right rotation. [5 marks]

Topics: 05 - AVL-Trees

4. For each of the following functions to calculate the largest element of an array, work out the formula for the number of operations (assignments, comparisons and arithmetic operations) executed as a function of the size of the array, n , state which complexity class in big O notation the function belongs to and justify your answer with a short explanation. Assume that the sort algorithm used in (c) takes $O(n \log n)$ operations.

```
(a) int largest1(int [] arr) {  
2   int n = arr.length;  
3   int max = 0;  
4   for (int i=0; i<n; i++) {  
5       bool largest = true;  
6       for (int j=0; j<n; j++) {  
7           if (arr[i] < arr[j])  
8               largest = false;  
9       }  
10      if (largest)  
11          max = arr[i];  
12  }  
13  return max;  
14 }
```

[7 marks]

```
(b) int largest2(int [] arr) {  
2   int n = arr.length;  
3   int max = 0;  
4   if (arr.length == 0) {  
5       return 0;  
6   } else {  
7       max = arr[0];  
8       for (int i=0; i<n; i++) {  
9           if (arr[i] > max)  
10              max = arr[i];  
11      }  
12      return max;  
13  }  
14 }
```

[7 marks]

```
(c) int largest3(int [] arr) {  
2   sort(arr);  
3   if (arr.length == 0) {  
4       return 0;  
5   } else {  
6       int last = arr[arr.length - 1];  
7       return last;  
8   }  
9 }
```

[6 marks]

Topics: : 03 - Efficiency and Complexity

5. Consider a binary search tree used to store integers with methods `isEmpty(t)`, `left(t)`, `right(t)` and `root(t)`, to return whether the tree `t`, is empty, the left child tree, the right child tree and the integer value stored at the root respectively.

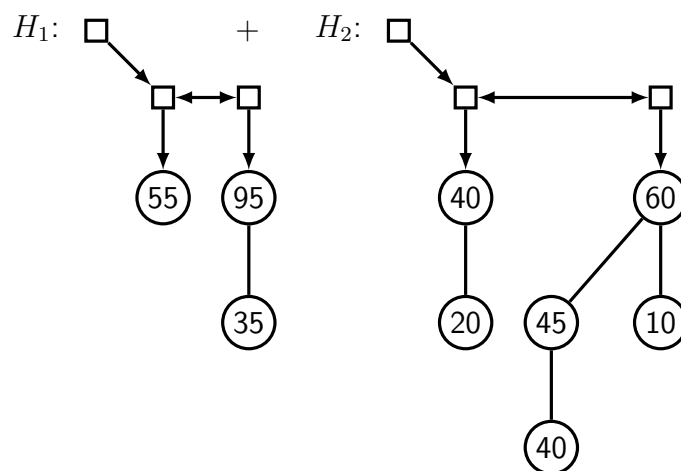
- Write a **recursive** function `sum_rec(tree)` to calculate and return the sum of all integers stored in the tree. **[8 marks]**
- A `Queue` ADT has, for a queue `q`, methods `q.enqueue(val)` and `q.dequeue()` to enqueue a value `val` in the queue and to dequeue and return a value from the queue respectively. It also has a constructor which can be invoked with the command `new Queue()` to create and return a new empty queue.

Write the pseudocode for a **non-recursive** function `sum_nonrec(tree)`, to calculate and return the sum of all integers stored in the tree. **[12 marks]**

Topics: 02 - Stacks and Queues, 04 - Trees

6. (i) Consider a Binary Heap where the highest priority is the highest value. The standard Binary Heap `heapify` method is used to construct this Binary Heap on an Array that contains, in index locations 1 to 10, the integers 1 to 10. Choose and write out an ordering of these integers in the array that **maximises** the number of swaps that the `heapify` method will execute on the array and report the total number of swaps executed. **[10 marks]**

(ii) Draw the result of merging the following two Binomial Heaps:



[10 marks]

Topics: 07 - Priority Queues and Heap Trees

Model answer

1.

- (a) Have two stacks, `in` and `out`. In order to enqueue a value, iterate, popping each element off the `out` stack and pushing it onto the `in` stack until the `out` stack is empty. Then push the new value onto the `in` stack. When you want to dequeue a value, iterate, popping each element off the `in` stack and pushing it onto the `out` stack until the `in` stack is empty. Then obtain the value to be dequeued by popping it off the `out` stack.
- (b) [Note: any reasonable pseudocode ADT for stacks can be used so long as it is used consistently. Here I assume an object oriented ADT such as might be written in Java, but you can use the ADT in the handouts if you prefer.]

```
1 enqueue(val)
2 {
3     while (not out.isEmpty())
4         in.push(out.pop())
5     in.push(val)
6 }
```

(c)

```
1 dequeue()
2 {
3     while (not in.isEmpty())
4         out.push(in.pop())
5     if (out.isEmpty())
6         throw QueueEmptyException("Tried to dequeue from an empty queue")
7     return out.pop()
8 }
```

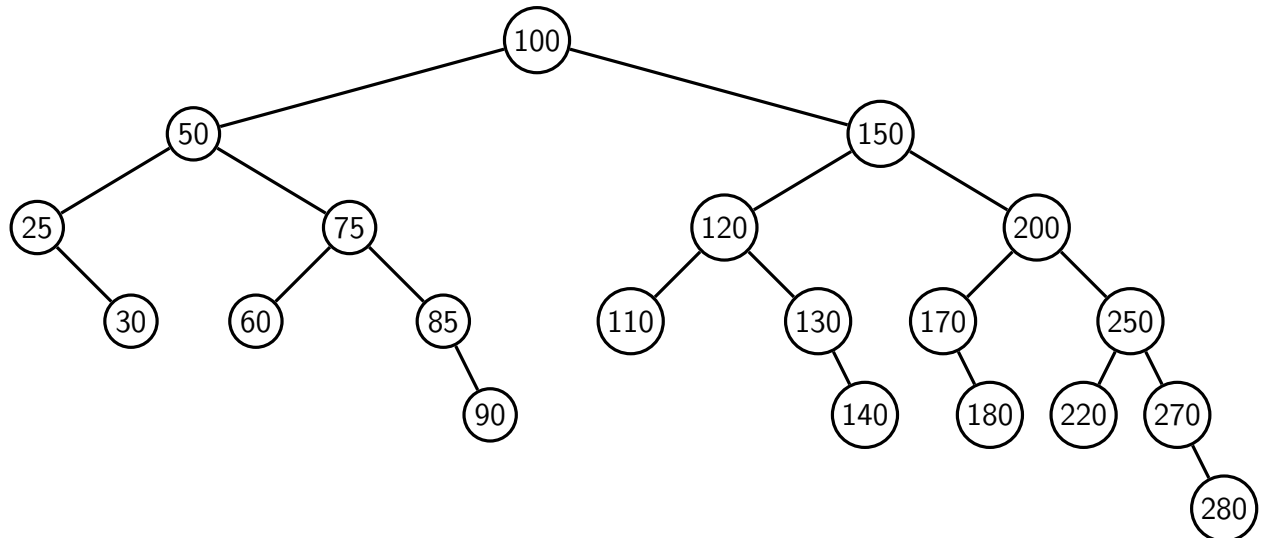
- (d) Here the cost of enqueue and dequeue are both $O(n)$ in the worst case, as you may have to transfer all elements from one stack onto the other to complete each operation. You can make it $O(1)$ for one of the operations by making one operation do the transfer, carry out its operation (enqueue or dequeue) and transfer back, leaving the other operation not needing to do any transfers between stacks. However, the amortised cost of enqueue and dequeue over a random sequence of such operations would remain $O(n)$

2. [There are multiple different solutions. This one allows pushing empty (sub-)trees onto the stack. The code becomes slightly longer but slightly more efficient if you only push non-empty (sub-)trees onto the stack.]

```
1 size(t)
2 {
3     Stack todo = new Stack()
4     todo.push(t)
5     size = 0
6
7     while (not todo.isEmpty())
8     {
9         tree = todo.pop()
10        if (not tree.isEmpty())
11        {
12            size++
13            todo.push(tree.left())
14            todo.push(tree.right())
15        }
16    }
17
18    return size
19 }
```

3.

- (a) [This is a minimal AVL tree of height 5. You can switch the left and right children of any node and it remains a minimal AVL tree (so long as you also correct the values in the nodes so that it observes the Binary Search Tree ordering requirements).]



- (b) Inserting the value 20 would not cause any rotation because it would be entered as a new left leaf of the node with value 25, and not cause any imbalance higher up the tree.
- (c) Inserting the value 95 would cause a single rotation. The new leaf would be inserted as the right child of the node with value 90, causing a -2 imbalance at the node containing 85. This corresponds to a RR case which would be fixed with a left rotation at 85.
- (d) Inserting the value 87 would cause a double rotation. The new leaf would be inserted as the left child of the node with value 90, causing a -2 imbalance at the node containing 85. This corresponds to a RL case which would be fixed with a right rotation at 90 followed by a left rotation at 85.

4. Justification should analyse the steps and the loops in the code

- (a) `largest1`: $O(n^2)$
- (a) `largest2`: $O(n)$
- (a) `largest3`: The dominant complexity will be the sort, so $O(n \log n)$ by default but other complexities will be accepted if a sort with a different complexity is used, or simply saying the complexity is that of the sort.

5. (a)

```
1 sum_rec(t)
2 {
3     if (isEmpty(t) )
4         return 0
5     else
6         return sum_rec(left(t)) + root(t) + sum_rec(right(t))
7 }
```

(b)

```
1 sum_nonrec(t)
2 {
3     Queue todo = new Queue()
4     todo.enqueue(t)
5     sum = 0
6
7     while ( not todo.isEmpty() )
8     {
9         tree = todo.dequeue()
10        if ( not tree.isEmpty() )
11        {
12            sum += root(tree)
13            todo.enqueue(tree.left())
14            todo.enqueue(tree.right())
15        }
16    }
17    return sum
18 }
```


6. (i) There are many different orderings that will ensure that the maximum number of swaps are required, but the simplest is to have them in reverse priority order. Since the highest priority is the highest value, that means reverse priority order is increasing numeric value, i.e. $1, 2, 3, \dots, 10$.

With this ordering the heapify algorithm will iterate, starting at the last internal node and continuing up through all the internal nodes until finally arriving at the root node. At each iteration it will trigger a sequence of swaps so that that internal node will be swapped on a full path from its current location down to the leaf level to end up as a leaf node.

- (ii) The result must be exactly as follows to earn full marks

